

Building an NBA Game Model

The math, explained from scratch

Reference guide · October 2026

How to use this guide

Every symbol is explained the first time it appears. Section 0 covers the basic tools; sections 1–8 build the model piece by piece; the last page gives the build order.

Contents

0. A few basics first	2
1 Predict the score margin, not just who wins	2
2 Player ratings (the core)	3
2.1 How to estimate β : RAPM	4
2.2 Fitting with a prior (ridge regression)	4
3 Ratings that move over time (Kalman filter)	5
4 Projecting minutes (where the injury value comes from)	6
5 Score distribution → every market	7
6 Context terms	8
7 Tuning without fooling yourself	8
7.1 Scoring rules: how you grade a forecast	8
7.2 Validation: testing honestly	9
7.3 Hyperparameters and tuning	9
7.4 Don't fool yourself	9
7.5 Combining models	9
8 Measuring the gap to the market	10
What to build, in order	10

0. A few basics first

Probability (p). A number between 0 and 1 for how likely something is. $p = 0.6$ means “happens 60% of the time.”

Random variable. A quantity whose value you don’t know in advance, like tonight’s final margin. You describe it by how likely each value is.

Normal distribution, $\mathcal{N}(\mu, \sigma^2)$. The bell curve. Two numbers describe it:

- μ (mu), the **mean**: the center, or the “expected” value.
- σ (sigma), the **standard deviation**: how spread out the outcomes are. About 68% of outcomes land within $\pm\sigma$ of the mean, and about 95% within $\pm 2\sigma$.
- σ^2 is the **variance**: the standard deviation squared. Variances are convenient because independent sources of randomness add up as variances.

$X \sim \mathcal{N}(\mu, \sigma^2)$ reads “ X is distributed as a normal with mean μ and variance σ^2 .”

$\Phi(z)$ (capital phi). The probability that a standard bell curve (mean 0, standard deviation 1) comes out below z . For example, $\Phi(0) = 0.5$, $\Phi(1) \approx 0.84$ and $\Phi(-1) \approx 0.16$.

$\mathbb{E}[X]$. The **expected value** of X : its long-run average if the situation were repeated many times.

$\sum_i$ (capital sigma). “Add up over all i .” For example, $\sum_{i=1}^3 x_i = x_1 + x_2 + x_3$.

Logit. A way of stretching probabilities out to the whole number line:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

Where

- $\frac{p}{1-p}$ is the **odds**. For $p = 0.75$ the odds are 3 (“3 to 1”).
- $\ln$ is the natural logarithm.
- $\text{logit}(0.5) = 0$. Probabilities above 50% give positive numbers and below 50% negative ones.

You average or adjust probabilities in logit space because a step near 50% and a step near 95% mean very different things in raw probability.

1. Predict the score margin, not just who wins

Margin. Home points minus away points. +7 means the home team won by 7.

Why bother: a win/loss result treats a 30-point win and a 1-point win the same, so it throws away information. The margin keeps it.

$$\text{margin} = (r_{\text{home}} - r_{\text{away}}) + h + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Where

- $r_{\text{home}}, r_{\text{away}}$: each team’s **rating**, measured in points. If $r_{\text{home}} = +5$ and $r_{\text{away}} = +1$, home is 4 points better on a neutral court.
- h : **home-court advantage**, the extra points a team gets just for playing at home (about 2 in today’s NBA).
- ε (epsilon): the **noise**, everything you can’t predict (hot shooting, refs, luck). It has mean 0

because luck doesn't favor either side on average.

- σ : how big that luck typically is. In the NBA, $\sigma \approx 12$ –14 points.

Your **expected margin** is the noise-free part, $\mu = r_{\text{home}} - r_{\text{away}} + h$. To win, the actual margin has to be above 0:

$$P(\text{home win}) = P(\text{margin} > 0) = \Phi\left(\frac{\mu}{\sigma}\right)$$

Example: $\mu = 6$ and $\sigma = 13$ give $\Phi(0.46) \approx 0.68$, a 68% home win probability.

Two refinements:

- **Damp blowouts.** “Garbage time” is the end of a game that’s already decided, when both teams play backups. It adds noise, so cap big margins (for example at 20), use a log scale, or use the margin at the moment the game was decided.
- **Make σ depend on the game.**
 - A **possession** is one team’s turn with the ball, ending in a shot, turnover or foul. More possessions means more random events, so $\sigma \propto \sqrt{\text{possessions}}$. The symbol $\propto$ means “proportional to.”
 - Your ratings are estimates and could be wrong. That uncertainty adds to the game’s own randomness:

$$\sigma_{\text{total}}^2 = \sigma_{\text{game}}^2 + \text{Var}(\mu)$$

Where

- σ_{game}^2 : the game’s own randomness.
- $\text{Var}(\mu)$: how uncertain you are about the expected margin itself.

Key idea

Early in the season $\text{Var}(\mu)$ is large, so σ_{total} is wider and probabilities sit closer to 50%. That’s the honest way to express “we don’t know these teams yet.”

2. Player ratings (the core)

Instead of rating teams, rate **players** and build each team from whoever plays tonight:

$$r_{\text{team}} = \sum_i \frac{m_i}{48} \beta_i$$

Where

- i : each player on tonight’s roster.
- m_i : the minutes player i is expected to play. The team’s minutes add up to $5 \times 48 = 240$.
- $\frac{m_i}{48}$: the share of the game he’s on the floor, since an NBA game lasts 48 minutes.
- β_i (beta): player i ’s **impact rating**: how many points per 100 possessions his team gains when he’s on the floor rather than an average player. A star is about +6; a bench player can be −2.

How to estimate β : RAPM

RAPM (Regularized Adjusted Plus-Minus) asks one question: when this player is on the floor, does his team outscore the opponent by more than you'd expect from the other nine players out there?

A **stint** is a stretch of a game when the same 10 players are on the floor. Every substitution starts a new one. A season has roughly 30,000 stints. For each stint s :

$$y_s = 100 \cdot (\text{PPP}_{\text{off}} - \text{PPP}_{\text{def}})$$

Where

- **PPP**: points per possession. PPP_{off} is how well one team scored during the stint and PPP_{def} is how well the other team scored.
- Multiplying by 100 makes it “points per 100 possessions,” the standard unit.
- y_s is the stint's **outcome**: how much better one lineup was than the other.

The model says that outcome equals the sum of the players' ratings:

$$y_s = \sum_{i \in \text{off}} \beta_i - \sum_{j \in \text{def}} \beta_j + \text{HCA} + \varepsilon_s$$

Where

- $i \in \text{off}$: “for each player i in the offensive five”; $j \in \text{def}$: the defensive five.
- **HCA**: home-court advantage again.
- ε_s : the stint's luck.

This is a **regression**: finding the numbers (β) that make the predicted outcomes match the actual outcomes as closely as possible. In matrix form, X_s is a row with +1 for each offensive player, -1 for each defensive player and 0 for everyone else, so $X_s\beta$ is the predicted outcome for stint s .

Fitting with a prior (ridge regression)

Plain regression goes wrong here. Teammates who always play together can't be told apart, and players with few minutes get wild ratings from small samples. The fix is to pull every rating toward a reasonable starting guess:

$$\hat{\beta} = \arg \min_{\beta} \underbrace{\sum_s w_s (y_s - X_s \beta)^2}_{\text{fit the data}} + \underbrace{\lambda \|\beta - \beta_{\text{prior}}\|^2}_{\text{stay near the prior}}$$

Where

- $\hat{\beta}$ (“beta hat”): the **estimate**, your best guess at the ratings.
- $\arg \min_{\beta}$: “the values of β that make the following expression as small as possible.”
- $(y_s - X_s \beta)^2$: the **squared error**, how wrong the prediction was for that stint, squared so misses in either direction count and big misses count more.
- w_s : the stint's **weight**, its number of possessions. A 12-possession stint tells you more than a 2-possession one.
- β_{prior} : the **prior**, your guess before looking at the stint data (see below).

- $\|\beta - \beta_{\text{prior}}\|^2$: how far the ratings move from the prior, summed as squared distances over all players.
- λ (lambda): how strongly to stay near the prior. A large λ trusts the prior; a small λ trusts the stint data. You tune it (section 7).

This penalized version of regression is called **ridge regression**. The effect: a player with 3,000 minutes gets a rating driven mostly by the data, and a player with 50 minutes stays close to his prior.

Where the prior comes from:

- **Box-score prior.** The box score is the standard stat line (points, rebounds, assists, steals, blocks, turnovers). Regress past RAPM values on these stats per 100 possessions to learn which stats predict impact, then apply that to each player. This is what **BPM** (Box Plus-Minus) does. Adjust for age, since players typically improve until about 27 and decline after about 30.
- **Rookies** have no NBA stats, so their prior comes from draft position.

Two more refinements:

- **Offense and defense separately.** Split each player into β^{off} (how much he helps his team score) and β^{def} (how much he stops the opponent). This is also what makes a totals model possible.
- **Time decay.** Recent games matter more:

$$w_s \propto e^{-\text{age}_s/\tau}$$

Where

- age_s : how many days ago stint s happened.
- e^{-x} : exponential decay, which shrinks smoothly from 1 toward 0.
- τ (tau): how fast old data fades. After τ days a stint counts about 37% as much; after 2τ , about 14%.

Public models like **EPM** and **DARKO** work this way.

3. Ratings that move over time (Kalman filter)

Players change during a season: injuries, slumps, development. A **Kalman filter** is a method for tracking a quantity you can't see directly from noisy measurements. It also keeps track of how sure it is.

State equation (how the true rating drifts):

$$\beta_t = \beta_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q)$$

Where

- t : the current game (or day); $t - 1$ is the previous one.
- β_t : the true rating at time t , which you never observe directly.
- η_t (eta): a small random change in true ability.
- Q : how much ratings can drift per step. Set it larger in the offseason and after trades, when

ability can really change.

Observation equation (what a game shows you):

$$y_t = X_t\beta_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R)$$

Where

- y_t : what you saw (the margin or stint outcome).
- X_t : who played.
- ε_t : game luck, with variance R , which is how noisy one game is.

The filter stores two things per rating: your best guess and its uncertainty P_t (a variance). After each game it moves the guess toward what it saw:

$$\text{new guess} = \text{old guess} + K_t \cdot (\text{what happened} - \text{what you predicted}), \quad K_t = \frac{P_t}{P_t + R}$$

Where

- K_t : the **Kalman gain**, between 0 and 1, which says how much to react.
- If you're very unsure (P_t large compared with R), K_t is near 1 and you react strongly.
- If you're confident (P_t small), K_t is near 0 and one game barely moves the rating.
- After each update P_t shrinks, because you've learned something. Between games it grows by Q , because ability may have drifted.

Key idea

This is exactly Elo's K factor, except it adjusts itself. Your Elo uses a fixed $K = 10$ for every team at every point in the season, a special case of this.

4. Projecting minutes (where the injury value comes from)

Player ratings only help if you know who will actually play, and for how long:

$$\mathbb{E}[\text{min}_i] = P(\text{plays}_i \mid \text{status}_i) \cdot \mathbb{E}[\text{min}_i \mid \text{plays}]$$

Where

- $\mathbb{E}[\text{min}_i]$: player i 's expected minutes tonight.
- $P(A \mid B)$: a **conditional probability**, "probability of A given B." Here: how likely he plays, given his injury-report status (Out, Doubtful, Questionable, Probable, not listed). You already estimated these from the archive.
- $\mathbb{E}[\text{min}_i \mid \text{plays}]$: how many minutes he gets if he does play, for example the average of his recent games.

Redistribution. When a player sits, his minutes don't disappear; teammates absorb them. Spread his minutes across teammates using how they were shared historically when he was out, mostly to players at the same position. The team rating then reflects the real replacements, not just a missing star.

Key idea

Getting minutes right matters about as much as getting ratings right: a perfect β times the wrong minutes is still wrong.

5. Score distribution $\rightarrow$ every market

Instead of predicting only the margin, predict **both teams' scores**. Then spreads, moneylines and totals all come from one model.

Ratings, in points per 100 possessions:

- **ORtg: offensive rating**, points a team scores per 100 possessions.
- **DRtg: defensive rating**, points it allows per 100 possessions.
- **lgRtg**: the league-average rating, the reference point.

Expected points:

$$\text{Pts}_{\text{home}} = \text{Poss} \cdot \frac{\text{ORtg}_{\text{home}} \cdot \text{DRtg}_{\text{away}}}{100 \cdot \text{lgRtg}}, \quad \text{Pts}_{\text{away}} = \text{Poss} \cdot \frac{\text{ORtg}_{\text{away}} \cdot \text{DRtg}_{\text{home}}}{100 \cdot \text{lgRtg}}$$

In words: a good offense against a bad defense scores more than either number alone suggests. Dividing by lgRtg keeps the scale right: an average offense against an average defense scores exactly the league average.

Possessions:

$$\text{Poss} = f(\text{pace}_{\text{home}}, \text{pace}_{\text{away}})$$

Where

- **Pace**: how many possessions a team usually plays per 48 minutes. Fast teams run, slow teams grind.
- $f(\cdot)$: some function combining the two. A simple version is the average of both teams' paces, adjusted to the league average.

Both scores together (bivariate normal):

$$\begin{pmatrix} \text{Pts}_{\text{home}} \\ \text{Pts}_{\text{away}} \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_H \\ \mu_A \end{pmatrix}, \begin{pmatrix} \sigma_H^2 & \rho \sigma_H \sigma_A \\ \rho \sigma_H \sigma_A & \sigma_A^2 \end{pmatrix}\right)$$

Where

- **Bivariate normal**: a bell curve for two quantities at once.
- μ_H, μ_A : expected home and away points (from the formulas above).
- σ_H, σ_A : how much each team's score typically varies (about 11 points).
- The 2×2 grid is the **covariance matrix**. The diagonal holds each score's variance, and the off-diagonal says how the two scores move together.
- ρ (rho): the **correlation** between the two scores, from -1 to $+1$. It's small and positive because a fast game, or one with lots of free throws, raises both teams' scores.

From this you get:

$$\begin{aligned} \text{Margin} &\sim \mathcal{N}(\mu_H - \mu_A, \sigma_H^2 + \sigma_A^2 - 2\rho \sigma_H \sigma_A) \\ \text{Total} &\sim \mathcal{N}(\mu_H + \mu_A, \sigma_H^2 + \sigma_A^2 + 2\rho \sigma_H \sigma_A) \end{aligned}$$

Where

- **Margin** (for moneylines and spreads): when the scores move together, the margin varies *less*, hence the -2ρ .
- **Total** (for over/unders): when they move together, the total varies *more*, hence the $+2\rho$.

Key idea

Because both come from one distribution, your spread, moneyline and total prices are always consistent with each other.

6. Context terms

Context terms are extra adjustments for the situation of a game, added to the expected margin:

- **Home court**: playing at home.
- **Rest and back-to-backs**: a back-to-back is playing on two consecutive days. Tired legs cost points.
- **Altitude**: Denver and Utah are high up, and visitors tire faster.
- **Travel**: long trips and time-zone changes.
- **Motivation**: late in the season some teams are chasing a playoff spot and others are **tanking** (losing on purpose for a better draft pick).

Fit them **jointly**, meaning estimated at the same time as the ratings, so the model doesn't count the same effect twice. For example, if Denver's rating already includes its home altitude advantage, a separate altitude term would double it.

Your audit already showed these explain little once ratings are good, so keep only a few.

7. Tuning without fooling yourself

Scoring rules: how you grade a forecast

A **scoring rule** turns each forecast and its outcome into a number for how good the forecast was. The ones below are **proper**: you get the best score only by reporting the probabilities you actually believe, so they reward accuracy, not bold guessing.

$$\text{LogLoss} = -\frac{1}{N} \sum_{g=1}^N \left[y_g \log p_g + (1 - y_g) \log(1 - p_g) \right], \quad \text{Brier} = \frac{1}{N} \sum_{g=1}^N (p_g - y_g)^2$$

Where

- N : the number of games; g : one game.
- p_g : your predicted probability that the home team wins game g .
- y_g : what happened, 1 if home won and 0 if not.
- **Brier score**: the average squared gap between your probability and the result. Lower is better. Always saying 50% scores 0.25, and your model scores about 0.20.
- **Log loss**: the average "surprise." Lower is better. It punishes confident wrong forecasts very hard: saying 95% and losing costs a lot. Use it as the main score, with Brier as a check.

Validation: testing honestly

- **Out-of-sample:** testing on games the model never saw during fitting. That’s the only honest test.
- **Walk-forward:** train on seasons 1–3, test on season 4; then train on 1–4, test on 5. It mimics real life, where you only ever know the past.
- **No lookahead:** every input must have been known before the game, such as injury reports issued before $T - 60$. You already enforce this.

Hyperparameters and tuning

- **Hyperparameters** are settings you choose rather than learn from the data directly: λ , τ , Q , σ and the context-term sizes.
- **Optuna** is a Python library that searches for good hyperparameters with **Bayesian optimization**: it tries settings, learns which areas look promising, and tries there next. That’s much faster than testing every combination.

Don’t fool yourself

- **Overfitting:** when a model learns the noise in its training data instead of the real pattern. It looks great in testing and fails live.
- **Best-of- N inflation:** if you try 200 variants and keep the best, part of its score is luck. The more you try, the more the winner’s score is inflated.
- Defenses:
 - Cap the number of trials.
 - Keep one **holdout season** you never look at during development, and test on it exactly once at the end.
 - Keep your **paired bootstrap** rule. A bootstrap resamples your games many times to see how much a result could change by chance. “Paired” means comparing two models on the same resampled games. If the 95% range of the improvement includes 0, the improvement isn’t proven.

Combining models

Different models make different mistakes, so blending them helps. This is called **ensembling**:

$$\text{logit}(p_{\text{blend}}) = \sum_k w_k \text{logit}(p_k)$$

Where

- k : each model (Elo, RAPM-based, box-score-only).
- p_k : model k ’s probability.
- w_k : how much to trust model k , fitted from data.
- Blending in logit space (section 0) behaves better than averaging raw probabilities.

Fit the weights on **out-of-fold predictions**: predictions each model made on games it wasn’t trained on. Otherwise a model that memorized its training data looks better than it is and gets too much weight.

Gradient boosting (e.g. LightGBM) is a machine-learning method that builds many small decision trees, each correcting the previous ones' errors. It's powerful but easily overfits on about 1,200 games a season. So use it only on the **residuals** (what the main model still gets wrong), and with **monotonic constraints**: rules like "more rest can never hurt a team," which stop the model from learning nonsense patterns.

8. Measuring the gap to the market

The **closing line** is the betting market's final price right before tip-off. It includes all public information, including late injury news, so it's the strongest benchmark there is.

Forecast (2025-26, 974 games)	Brier
DraftKings closing line	0.1948
DraftKings opening line	0.1998
Your model (v3)	0.2033

Split the gap into two parts:

- **Information gap**: games where the injury report changed late (after $T - 60$, your cutoff). The market knew something your model didn't yet. Better math can't fix that; only faster information could.
- **Model gap**: everything else. That's how much worse your probabilities are with the same information. Better modeling can close this part.

Realistic target

A very good public model reaches about DraftKings' **opening line** (the first price posted, days before the game), 0.1998 on your data. Beating the close is something even professional syndicates rarely do on NBA sides.

What to build, in order

1. **Margin target** (section 1): the cheapest upgrade.
2. **RAPM with a box-score prior** (section 2): the biggest expected gain.
3. **Minutes projections** (section 4): makes player ratings usable.
4. **Kalman filter** (section 3): ratings that adapt with honest uncertainty.
5. **Bivariate score model** (section 5): unlocks totals.
6. **Model blending** (section 7): the final polish.

Steps 2 and 3 together are where most of the improvement is likely to be.

Symbol cheat sheet

Symbol	Meaning	Symbol	Meaning
p	probability	β_i	player impact (pts/100 poss)
μ	mean / expected value	λ	ridge strength (pull to prior)
σ, σ^2	std. deviation, variance	τ	time-decay speed
$\Phi(z)$	P(standard normal $< z$)	Q	rating drift per step
$\mathbb{E}[X]$	expected value of X	R	noise of one game
$\text{logit}(p)$	$\ln \frac{p}{1-p}$	K_t	Kalman gain (how much to react)
ε, η	random noise terms	P_t	uncertainty of a rating
h, HCA	home-court advantage	ρ	correlation of the two scores
m_i	player minutes	w	weight
$\propto$	proportional to	$\hat{\beta}$	estimated value